

Asociación y dependencia entre Clases

Los procesos de abstracción y clasificación permiten identificar, representar y agrupar a entidades que son semejantes de acuerdo a algún criterio. El criterio adoptado en una clasificación permite decidir si una entidad pertenece a una clase o no.

En la construcción de un modelo para una aerolínea seguramente se agrupen en una clase los vuelos que se realizan, en otra las ciudades a las que llegan los vuelos, otra clase estará conformada por los pilotos habilitados para comandar una nave y otra por las tarjetas de embarque. Las entidades de una clase son semejantes de acuerdo al criterio elegido y además se **relacionan** de la misma manera con las entidades de otras clases.

La **asociación** es una relación entre clases que se produce cuando el modelo de un objeto del problema **contiene** o puede contener al modelo de otro objeto del problema. Por ejemplo, todo vuelo **tiene** una ciudad como destino final y **puede tener** una o más ciudades en las que hace escala.

La relación de **dependencia** se produce cuando el modelo de un objeto del problema **usa** al modelo de otro objeto. Por ejemplo, un pasajero **usa** su tarjeta de embarque para despachar su equipaje.

En la programación orientada a objetos el punto de partida para la construcción de un sistema es un proceso de abstracción y clasificación. Los objetos de una clase se caracterizan por tener los mismos atributos y comportamiento, pero además comparten entre sí el mismo modo de relacionarse con objetos de otras clases.

Un objeto asociado a otro objeto, **tiene un** atributo de la misma u otra clase. Por ejemplo, un objeto de la clase Vuelo tiene un atributo destino de la clase Ciudad. La relación entre los objetos provoca una relación entre las clases, que se dicen **asociadas**.

Un objeto depende de un objeto de otra clase, si **usa un** objeto de esta otra clase. Por ejemplo, un objeto de la clase Pasajero usa un objeto de la clase TarjetaEmbarque. La relación entre los objetos provoca una relación de dependencia entre las clases.

Cuando una clase está asociada a otra clase, los cambios en la segunda pueden tener un impacto en la primera. El mismo impacto se produce si se modifica una clase de la cual depende otra. Uno de los objetivos de la programación orientada a objetos es reducir el impacto de estos cambios.

Dependencia entre clases

Cuando una clase declara una variable local o un parámetro de otra clase, decimos que existe una **dependencia** entre la primera y la segunda y surge de una relación del tipo **usaUn** entre los objetos.

Caso de Estudio: Fábrica de Juguetes

*En una fábrica de autos de juguete una parte de la producción la realizan **robots**. Cada robot tiene una carga de energía que se va consumiendo a medida que ejecuta las órdenes que recibe. Cada robot es capaz de conectarse de modo tal que se recargue su energía hasta su capacidad máxima de 5000 unidades. Esta acción puede ejecutarse ante una orden externa o puede iniciarla el robot mismo cuando su energía está por debajo de las 100 unidades.*

Cada robot cuenta con piezas de diferentes tipos: ruedas, ópticas y chasis. La cantidad de piezas se incrementa cuando un robot recibe una orden de abrir una **caja de piezas** y se decrementa cuando arma un vehículo. Cada caja tiene piezas de todos los tipos. Desarmar una caja cualquiera demanda 50 unidades de energía. Armar un auto consume 70 unidades de energía, 4 ruedas, 6 ópticas y 1 chasis. Es responsabilidad de cada servicio que consuma energía recargarla cuando corresponda.

Robot	Caja
<pre><<Atributos de clase>> energiaMaxima: 5000 energiaMinima: 100 <<Atributos de instancia>> nroSerie: entero energia: entero ruedas: entero opticas: entero chasis: entero</pre>	<pre><<Atributos de instancia>> ruedas: entero opticas: entero chasis: entero</pre>
<pre><<Constructor>> Robot(nro:entero,caja:Caja) <<Comandos>> abrirCaja(caja: Caja) recargar() armarAuto() <<Consultas>> obtenerEnergia(): entero obtenerChasis(): entero obtenerRuedas(): entero obtenerOpticas(): entero cantAutos(): entero clone(): Robot equals(Robot r): boolean</pre>	<pre><<Constructor>> Caja(r,o,ch: entero) <<Comandos>> establecerRuedas(n:entero) establecerOpticas(n:entero) establecerChasis(n:entero) vaciar() <<Consultas>> obtenerChasis(): entero obtenerRuedas(): entero obtenerOpticas(): entero equals(c:Caja): boolean</pre>
<pre><<Responsabilidades>> Todos los servicios que consumen energía deciden recargar cuando energía es menor que la mínima.</pre>	

- `Robot(nro:entero,caja:Caja)` inicializa `energía` con el valor máximo, el número de serie con el primer parámetro y el resto de los atributos de acuerdo a las piezas del atributo `caja`, luego la vacía. Requiere `caja` ligada.
- `recargar()` recarga la energía del robot hasta llegar al máximo.
- `abrirCaja(caja:Caja)` aumenta las piezas disponibles de acuerdo a las cantidades de la caja y la vacía. Requiere `caja` ligada.
- `armarAuto()` decrementa las piezas disponibles. Requiere que haya piezas disponibles para armar un auto.
- `cantAutos():entero` retorna la cantidad de autos que puede armar el robot con las piezas que tiene disponibles, sin desarmar una caja.
- `equals(Robot r):boolean` Requiere `r` ligado.

Como siempre, los valores de los atributos de instancia se establecen en la creación del objeto y se modifican cuando se ejecutan los comandos.

La implementación parcial de `Robot` será:

```
class Robot {
//Atributos de clase
private static final int energiaMaxima = 5000;
private static final int energiaMinima = 100;
//Atributos de instancia
private int nroSerie;
private int energia;
private int ruedas;
private int opticas;
private int chasis;
//Constructor
public Robot (int nro,Caja caja){
/* inicializa energia con el valor máximo, el número de serie con el
primer parámetro y el resto de los atributos de acuerdo a las piezas
del atributo caja y luego la vacía. Requiere caja ligada.*/
    nroSerie = nro;
    energia=energiaMaxima;
    ruedas = caja.obtenerRuedas();
    opticas = caja.obtenerOpticas();
    chasis = caja.obtenerChasis();
    caja.vaciar();}
//Comandos
public void recargar(){
//recarga la energía del robot hasta llegar al máximo.
    energia=energiaMaxima;}
public void armarAuto() {
/*Requiere que se haya controlado si hay piezas disponibles*/
    ruedas -= 4 ;
    opticas -=6;
    energia -= 70;
    chasis --;
//Controla si es necesario recargar energía
    if (energia < energiaMinima)
        this.recargar(); }
public void abrirCaja (Caja caja) {
/*Aumenta sus cantidades según las de la caja y la vacía. Requiere
caja ligada*/
    ruedas += caja.obtenerRuedas();
    opticas += caja.obtenerOpticas();
    chasis += caja.obtenerChasis();
    energia -= 50;
    caja.vaciar();
/*Controla si es necesario recargar energía*/
    if (energia < energiaMinima)
        this.recargar();}
//Consultas
public int obtenerRuedas(){
    return ruedas;}
public int obtenerOpticas(){
    return opticas;}
public int obtenerChasis(){
```

```

    return chasis;}
public int obtenerNroSerie(){
    return nroSerie;}
public int obtenerEnergia(){
    return energia;}
public String toString(){
    return nroSerie+" "+ruedas+" "+opticas+" "+chasis;}}

```

Entre las clases `Robot` y `Caja` se establece una relación de **dependencia**. Un parámetro de un servicio provisto por la clase `Robot`, es de tipo `Caja`.

El constructor y el comando `abrirCaja(Caja caja)` provocan un **efecto colateral** sobre el parámetro `caja`. Es decir, no sólo modifica el estado interno del robot que recibe el mensaje, sino también sobre la caja que pasa como parámetro. Si se modificara la signature de los servicios provistos por la clase `Caja`, los cambios impactarían en la clase `Robot`.

La clase `FabricaJuguetes` usa los servicios provistos por `Robot`:

```

class FabricaJuguetes{
...
public void producir() {
    Caja caja = new Caja(100,150,25);
    Robot unRobot;
    unRobot = new Robot(111,caja);
...
    System.out.println(caja.obtenerRuedas());
...
    if (unRobot.cantAutos() == 0)
        unRobot.abrirCaja(new Caja(200,300,50));
    unRobot.armarAuto();
...}}

```

Los atributos, los comandos y las consultas de la clase `Caja` tienen los mismos nombres que algunos atributos y servicios de la clase `Robot`, cuando un objeto recibe un mensaje, su clase determina el método que va a ejecutarse.

Por ejemplo, en `caja.obtenerRuedas()` el mensaje `obtenerRuedas()` lo recibe un objeto de clase `Caja`, de modo que se ejecuta el método provisto por esa clase. Si en el método `producir()` de la clase `FabricaJuguetes`, el objeto `unRobot` recibe el mensaje `obtenerRuedas()` se ejecuta el método provisto por la clase `Robot`, porque `unRobot` es de clase `Robot`.

La clase `FabricaJuguetes` **depende** de las clases `Robot` y `Caja` porque declara variables locales de esas clases.

Asociación entre clases

Cuando una clase **tiene un atributo** de otra clase, ambas clases están **asociadas** y la relación es de tipo **tieneUn**. En general, entre dos clases asociadas también hay una relación de dependencia, algunos de los servicios provistos por una clase recibirán parámetros o retornarán un resultado de la clase asociada. Así, una relación de tipo **tieneUn** con frecuencia provoca una relación de tipo **usaUn**.

Caso de Estudio: Control de los signos vitales

Los signos vitales son medidas de variaciones fisiológicas que permiten valorar las funciones corporales básicas. Dos de los principales signos vitales son la temperatura corporal y la presión arterial. Se considera que existe un principio de alarma cuando estos valores escapan de los umbrales establecidos. Los valores de la presión sanguínea se expresan en kilopascales (kPa) o en milímetros del mercurio (mmHg). Para convertir de mmHg a kPa el valor se multiplica por 0,13. La temperatura se mide en centígrados.

PresionArterial	SignosVitales
<<Atributos de clase>> umbralMax, umbralMin:real <<Atributos de instancia>> maxima,minima:real	<<Atributos de clase>> umbralTemp:real <<Atributos de instancia>> temperatura: real presion:PresionArterial
<<Constructor>> PresionArterial (ma,mi:real) <<Consultas>> obtenerMaxima():real obtenerMinima():real obtenerMaximaKP():real obtenerMinimaKP():real obtenerPresionPulso():real alarmaHipertension():boolean	<<Constructor>> SignosVitales (t:real, p:PresionArterial) <<Consultas>> obtenerTemperatura():real obtenerPresion():PresionArterial alarma():boolean
Los valores están expresados en milímetros de mercurio	Las temperaturas están expresada en grados.

En PresionArterial

- `PresionArterial (ma,mi:real)` Requiere `ma > mi > 0`

En SignosVitales

- `SignosVitales (t:real,p:PresionArterial)` Requiere `p` ligada y `t > 0`

La funcionalidad de la consulta `alarma` está establecida en este caso en el texto que precede al diagrama.

El **estado interno** de cualquier objeto de clase `PresionArterial` mantiene los valores de dos atributos de instancia, `maxima` y `minima`, representados en milímetros de mercurio. La clase `PresionArterial` fue implementada en el capítulo anterior. La implementación de `SignosVitales` es:

```
class SignosVitales{
//Atributos de clase
//Las temperaturas están expresada en grados
private static final float umbralTemp=38.0;
//Atributos de instancia
private float temperatura;
private PresionArterial presion;
//Constructor
public SignosVitales(float t, PresionArterial p){
//Requiere p ligada y t mayor a 0
    temperatura = t;
    presion = p;}
}
```

```
//Consultas
public float obtenerTemperatura() {
    return temperatura;}
public PresionArterial obtenerPresion() {
    return presion ;}
public boolean alarma() {
    return temperatura>umbralTemp || presion.alarmaHipertension();}
}
```

Las clases `SignosVitales` y `PresionArterial` están asociadas. La clase `SignosVitales` **tieneUn** atributo de clase `PresionArterial`. Además, `SignosVitales` **usaA** `PresionArterial`, su constructor recibe un parámetro de clase `PresionArterial` y la consulta `obtenerPresion()` retorna un resultado de esa misma clase.

El constructor requiere que el parámetro `p` esté ligado, todo registro de la presión arterial implica medir la temperatura y la presión. Si la clase que usa a `SignosVitales` no cumple con su responsabilidad y crea un objeto invocando al constructor con el segundo parámetro no ligado, cuando este objeto reciba el mensaje `alarma`, la ejecución terminará anormalmente.

La clase `SignosVitales` puede acceder a cualquiera de los miembros públicos de la clase `PresionArterial`. Los atributos han sido declarados como privados, por lo tanto no son accesibles.

La clase `Control`, implementada parcialmente, usa los servicios provistos por `SignosVitales` y `PresionArterial`:

```
class Control{
...
public void controlDiario(int mil,int ma1,
                        int mi2, int ma2,
                        float t1,float t2){
    PresionArterial p6Hs, p12Hs;
    SignosVitales sv6Hs,sv12Hs;
    p6Hs = new PresionArterial(mil,ma1);
    p12Hs= new PresionArterial(mil,ma2);
    sv6Hs = new SignosVitales(t1,p6Hs);
    sv12Hs = new SignosVitales(t2,p12Hs);
    if (sv6Hs.obtenerTemperatura() ==sv12Hs.obtenerTemperatura())
        System.out.println("Temperatura Estable");} }
```

Existe en este caso una relación de dependencia entre `Control` y las clases `SignosVitales` y `PresionArterial`.

Representación por referencia de clases asociadas

La representación a través de referencias surge para modelar la asociación entre clases, como se ilustra con el siguiente caso de estudio:

Caso de Estudio: Videojuego

En un videojuego algunos de los personajes son aliens. Los aliens tienen cierta cantidad de antenas y de manos, que determinan su capacidad sensora y su capacidad de lucha respectivamente.

Cada alien tiene un nombre y una cantidad de vidas que inicialmente es 5 y se van reduciendo cada vez que recibe una herida. Cuando está muerto ya no tienen efecto las heridas. Cuando un alien logra llegar a la base recupera 2 vidas, sin superar nunca el valor máximo de vidas, esto es, 5. El comando establecerVidas requiere un parámetro menor o igual al máximo de vidas.

Cada alien puede estar asociado a una nave. Cada nave tiene una velocidad y una cantidad de combustible en el tanque. Ambos atributos pueden aumentar o disminuir de acuerdo a un parámetro que puede ser positivo o negativo.

NaveEspacial	Alien
<pre><<Atributos de instancia>> velocidad:entero combustible:entero</pre>	<pre><<Atributos de clase>> maxVidas:entero <<Atributos de instancia>> nave:NaveEspacial vidas:entero antenas:entero manos:entero</pre>
<pre><<Constructor>> NaveEspacial(v,c:entero) <<Comandos>> cambiarVelocidad(v:entero) cambiarCombustible(c:entero) copy(n:NaveEspacial) <<Consultas>> obtenerVelocidad():entero obtenerCombustible():entero equals(n:NaveEspacial):boolean clone():NaveEspacial</pre>	<pre><<Constructor>> Alien(n:NaveEspacial,a,m:entero) <<Comandos>> recuperaVidas() recibeHerida() establecerVidas(v:entero) establecerNave(n:NaveEspacial) copy(a:Alien) <<Consultas>> obtenerNave():NaveEspacial obtenerVidas():entero obtenerAntenas():entero obtenerManos():entero obtenerPoder():entero equals(a:Alien):boolean clone():Alien</pre>

En la clase Alien:

- Alien (n:NaveEspacial,a,m:entero)
- establecerNave(n:NaveEspacial)
- establecerVidas(v:entero) requiere $v \leq \text{maxVidas}$.
- copy(a:Alien) y equals(a:Alien):boolean requieren a ligado.
- obtenerPoder():entero Si el alien tiene asociada una nave computa $(\text{antenas} + \text{manos}) * \text{vidas}$ sumado a la velocidad de su nave, en caso contrario computa $(\text{antenas} + \text{manos}) * \text{vidas}$

El diagrama de la clase NaveEspacial fue presentado en el capítulo anterior con las siguientes responsabilidades:

copy(n:NaveEspacial) y equals(n:NaveEspacial):boolean Requieren n ligado.

Las clases Alien y NaveEspacial están asociadas, la clase Alien tiene un atributo de clase NaveEspacial. En el problema, dos o más aliens pueden estar asociados a una misma nave

espacial. En la **solución**, dos o más objetos de clase **Alien** pueden mantener referencias a un mismo objeto de clase **NaveEspacial**. También es posible que dos naves tengan el mismo estado interno, pero distinta identidad. De acuerdo a este diseño un alien puede no estar ligado a una nave, lo cual impacta en su poder.

Una implementación para la **NaveEspacial** es:

```
class NaveEspacial {
//Atributos de instancia
private int velocidad;
private int combustible;
//Constructor
Public NaveEspacial(int v,int c){
    velocidad= v;
    combustible= c;}
//Comandos
public void cambiarVelocidad(int v){
    velocidad +=v;}
public void cambiarCombustible(int c){
    combustible +=c;}
public void copy (NaveEspacial n){
//Requiere n ligado
    velocidad=n.obtenerVelocidad();
    combustible=n.obtenerCombustible();}
//Consultas
public int obtenerVelocidad(){
    return velocidad;}
public int obtenerCombustible(){
    return combustible;}
public boolean equals (NaveEspacial n){
//Requiere n ligado
    return velocidad ==n.obtenerVelocidad() &&
        combustible==n.obtenerCombustible();}
public NaveEspacial clone(){
    return new NaveEspacial(velocidad, combustible);}}
```

La implementación de **Alien** de acuerdo al diseño propuesto:

```
class Alien {
//Atributos de clase
private static final int maxVidas=5;
//Atributos de instancia
private NaveEspacial nave;
private int vidas;
private int antenas;
private int manos;
//Constructor
public Alien (NaveEspacial n,int a,int m){
    nave = n;
    vidas = maxVidas;
    antenas = a;
    manos = m;}
//Comandos
public void establecerVidas(int v){
//Requiere v < maxVidas
    vidas = v;}}
```



```
public void establecerNave(NaveEspacial n){
    nave = n;}
public void recuperaVidas(){
    if (vidas+2 > maxVidas) vidas = maxVidas;
    else    vidas = vidas+2;}
public void recibeHerida(){
    if (vidas > 0) vidas--;}
public void copy (Alien a){
//Requiere a ligado
    nave = a.obtenerNave();
    vidas = a.obtenerVidas();
    antenas = a.obtenerAntenas();
    manos = a.obtenerManos();}
//Consultas
public NaveEspacial obtenerNave(){
    return nave;}
public int obtenerVidas(){
    return vidas;}
public int obtenerAntenas(){
    return antenas;}
public int obtenerManos(){
    return manos;}
public int obtenerPoder(){
    int p =(antenas+manos)*vidas;
    if (nave != null) p = p + nave.obtenerVelocidad();
    return p;}
public Alien clone(){
    NaveEspacial n = nave;
    int m = manos;
    int a = antenas;
    int v = vidas;
    Alien c = new Alien(n,a,m);
    c.establecerVidas(v);
    return c;}
public boolean equals(Alien a){
//Requiere a ligado
    return nave == a.obtenerNave() &&
        manos == a.obtenerManos() &&
        antenas == a.obtenerAntenas() &&
        vidas == a.obtenerVidas();}}
```

Cuando un alien recibe el mensaje `obtenerPoder()` envía el mensaje `obtenerVelocidad()` a su nave, de acuerdo a lo especificado por el diseñador.

El constructor y el comando `establecerNave` de la clase `Alien` reciben como parámetro una variable de la clase asociada `NaveEspacial`. Es responsabilidad de la clase que envía el mensaje `establecerNave` garantizar que el parámetro está ligado. Si no cumple con esta responsabilidad, es decir, si el atributo `nave` no está ligado, cuando el alien recibe el mensaje `obtenerPoder()` la ejecución terminará anormalmente en el momento que intente ejecutarse el método `obtenerVelocidad()`.

La consulta `obtenerNave` retorna una referencia a un objeto de la clase `NaveEspacial`. Se crea entonces una dependencia entre `Alien` y `NaveEspacial`, consecuencia de la

asociación entre clases. Si la signature de los métodos de la clase `NaveEspacial` cambia, la modificación puede afectar a la clase `Alien`.

A continuación *se asume* una **representación extendida** para objetos asociados y se analizan sus inconvenientes. Al ejecutarse el siguiente bloque de instrucciones del método `main`:

```
class testVideoJuego{
public static void main (String s[]){
    NaveEspacial n = new NaveEspacial(100,45);
    Alien a1,a2;
    a1 = new Alien(n,3,2);
    a2 = new Alien(n,2,3);
    a1.recibeHerida();
}
```

La Figura 1 muestra el diagrama de los objetos ligados a las variables `a1` y `a2`, para la representación extendida.

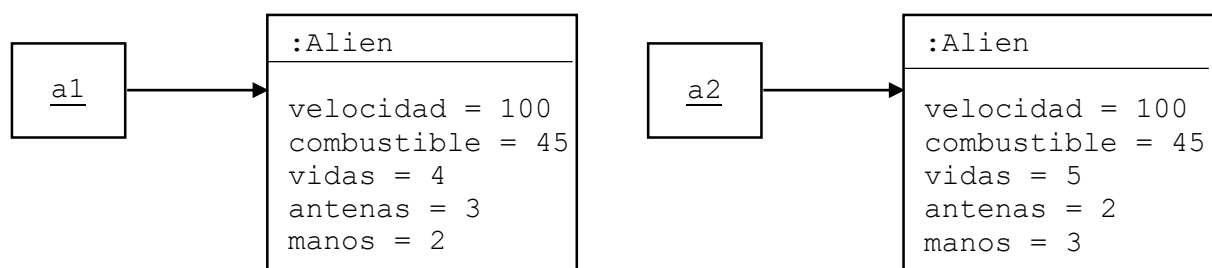


Figura 1

Es decir, el estado interno de un objeto de clase `Alien` mantiene los atributos de esa clase, más los atributos de la clase asociada `NaveEspacial`. Si dos o más objetos de la clase `Alien` están asociados a una misma instancia de la clase `NaveEspacial`, la estructura completa de cada objeto de la clase `NaveEspacial` se mantendrá en cada objeto de clase `Alien`.

La representación extendida provoca varios inconvenientes. En primer lugar, es necesario agregar un atributo a cada nave que permita identificarla, por ejemplo un código. En caso contrario, si los estados internos de dos objetos de clase `Alien` coinciden, no es posible decidir si están ligados a la misma nave o a naves diferentes que coinciden en la velocidad y en la cantidad de combustible.

En segundo lugar, cada vez que se modifica el valor de uno de los atributos que caracterizan a un objeto, es necesario modificar el estado interno de todos los objetos a los cuales está asociado. En este caso de estudio, cada vez que cambie la `velocidad` de una nave se debería modificar el estado interno de todos los aliens asociados a esa nave.

Otro inconveniente importante de la representación extendida es que si se modifica la representación de la clase `NaveEspacial`, por ejemplo, porque se agrega un atributo de instancia `motores`, este cambio va a afectar a la representación de todas las clases que usan los servicios de `NaveEspacial`. Los cambios en una clase provocan un impacto en la representación de los objetos de las clases asociadas.

Estos inconvenientes se solucionan usando una **representación por referencia**. El estado interno de un objeto contiene referencias a los objetos de las clases asociadas, de modo que un sistema complejo puede modelarse a partir de objetos simples. La modificación de una clase, no afecta la representación de los objetos de las clases asociadas.

Luego de la ejecución del mismo bloque de instrucciones, la Figura 2 muestra el diagrama de objetos, incluyendo al objeto ligado a la variable `n`, considerando una representación por referencia.

En este caso, si cambia la estructura del estado interno de un objeto de clase `NaveEspacial`, por ejemplo, porque se agregan nuevos atributos, no cambia la estructura del estado interno de los objetos de clase `Alien`. Si se modifican los valores de los atributos de una nave, por ejemplo, porque se reduce el combustible, no se modifican los valores de los atributos de los aliens que referencian a esa nave.

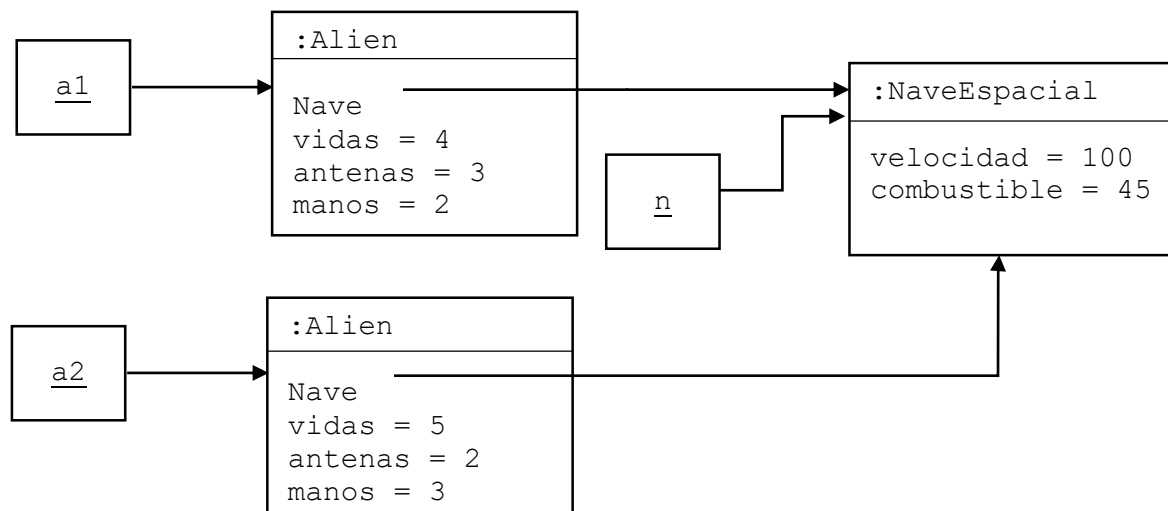


Figura 2

Igualdad y equivalencia entre objetos de clases asociadas

La representación por referencia afecta al diseño y la implementación de algunos servicios, en particular a los métodos `equals`, `copy` y `clone`. Cuando una clase está asociada a otra, la igualdad, copia y clonación se puede hacer en forma **superficial** o en **profundidad**.

Para decidir si dos objetos de una clase son iguales en forma superficial, al verificar que el estado interno de los objetos sean equivalentes se evalúa si están ligados a una misma instancia de la clase asociada. De manera análoga, la copia superficial modifica el estado interno del objeto que recibe el mensaje, con los valores almacenados en el objeto que pasa como parámetro. La clonación superficial retorna como resultado un objeto que tiene exactamente el mismo estado interno que el objeto que recibió el mensaje, incluyendo las referencias a objetos de las clases asociadas.

Cuando la igualdad es en profundidad la exigencia es menor, se requiere que los estados internos de los objetos asociados sean equivalentes, aunque las referencias sean distintas. En el caso de la copia en profundidad, el estado interno del objeto que recibe el mensaje se modifica con los valores de los atributos del objeto que se recibe como parámetro, excepto las referencias, que no se modifican, pero sí se modifica el estado interno de los objetos asociados. La clonación en profundidad retorna un nuevo objeto con el mismo estado interno que el objeto que recibe el mensaje, excepto las referencias que estarán ligadas a clones de los objetos asociados.

Para ilustrar la diferencia, se retoma el análisis del caso de estudio que asocia a las clases `Alien` y `NaveEspacial`. Los siguientes métodos implementan entonces **copia, clonación e igualdad superficial**:

```
public void copy (Alien a){
//Requiere a ligada
    nave = a.obtenerNave();
    vidas = a.obtenerVidas();
    antenas = a.obtenerAntenas();
    manos = a.obtenerManos();}
public Alien clone(){
    NaveEspacial n = nave;
    int m = manos;
    int a = antenas;
    int v = vidas;
    Alien c = new Alien(n,a,m);
    c.establecerVidas(v);
    return c;}
public boolean equals(Alien a){
//Requiere a ligada
    return nave == a.obtenerNave() &&
        manos == a.obtenerManos() &&
        antenas == a.obtenerAntenas() &&
        vidas == a.obtenerVidas();}
```

El comando `copy` copia el estado interno del objeto de clase `Alien` que se recibe como parámetro, en el estado interno del objeto de clase `Alien` que recibe el mensaje. Así, el valor del atributo `nave` del objeto ligado al parámetro `a`, se asigna al atributo `nave` del objeto que recibe el mensaje `copy`.

La consulta `clone` retorna un nuevo objeto de clase `Alien` que referencia a la misma nave que el objeto que recibió el mensaje. La siguiente implementación de `clone()` es equivalente a la anterior:

```
public Alien clone(){
    Alien c = new Alien(nave,antenas,manos);
    c.establecerVidas(vidas);
    return c;}
```

La consulta `equals` retorna `true` si los valores de los atributos de los objetos que se comparan son iguales, esto implica que mantienen referencias a una misma nave o bien no están ligadas.

Dada la siguiente clase tester:

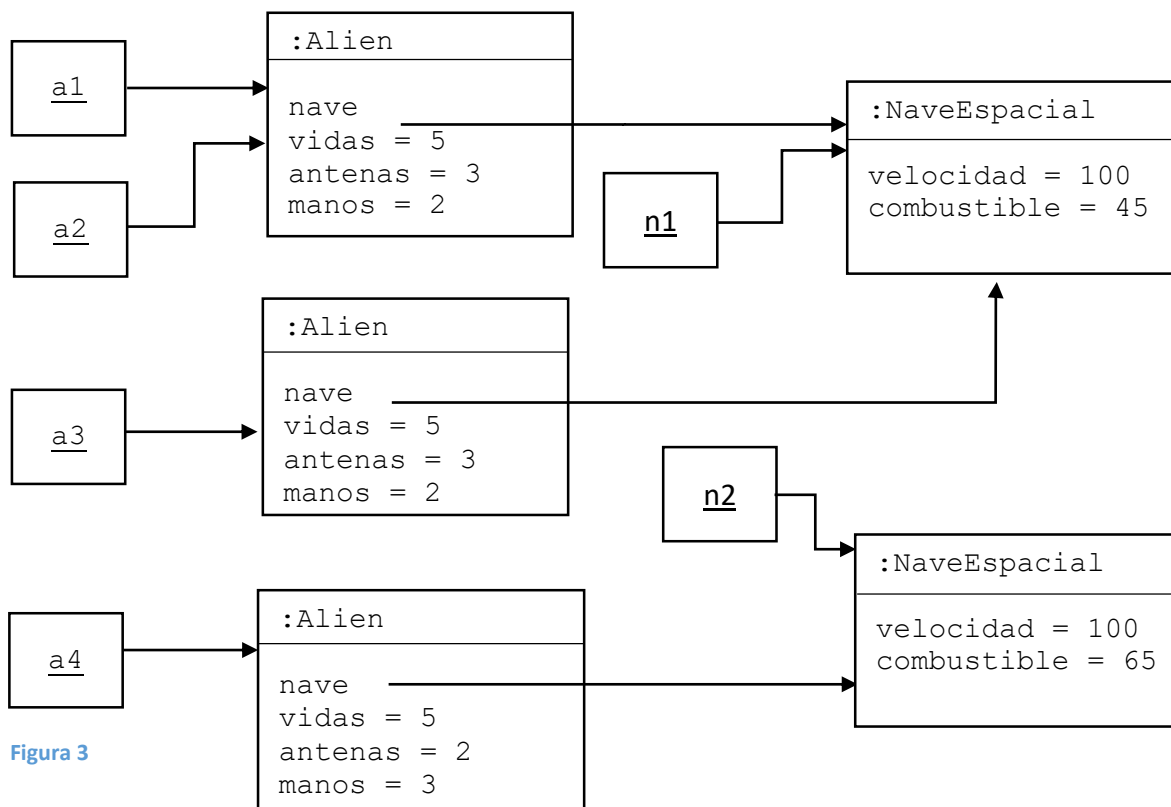
```
class testVideoJuego{
public static void main (String s[]){
    //Primer bloque
    NaveEspacial n1 = new NaveEspacial (100,45);
    NaveEspacial n2 = new NaveEspacial (100,65);
    Alien a1,a2,a3,a4;
    a1 = new Alien(n1,3,2);
    a2 = a1;
    a3 = a1.clone();
    a4 = new Alien(n2,2,3);
```

```

//Segundo bloque
a4.copy(a1);
System.out.println(a1==a2);
System.out.println(a1==a3);
System.out.println(a1==a4);
System.out.println(a1.equals((a2)));
System.out.println(a1.equals((a3)));
System.out.println(a1.equals((a4)));
System.out.println(a1==a2);
System.out.println(a1==a3);
System.out.println(a1==a4);
System.out.println(a1.equals((a2)));
System.out.println(a1.equals((a3)));
System.out.println(a1.equals((a4)));
//Tercer bloque
a1.recibeHerida();
a4.copy(a1);
System.out.println(a1.equals((a2)));
System.out.println(a1.equals((a3)));
System.out.println(a1.equals((a4)));
//Cuarto bloque
NaveEspacial n3 = n1.clone();
Alien a5=a1.clone();} }

```

La Figura 3 muestra el diagrama de objetos después de la ejecución de las instrucciones que siguen al comentario “Primer bloque”, considerando que clone se ha implementado en forma superficial.



Considerando que se implementa copy superficial, Figura 4 muestra el diagrama de objetos luego de la ejecución de:

```
a4.copy(a1);
```

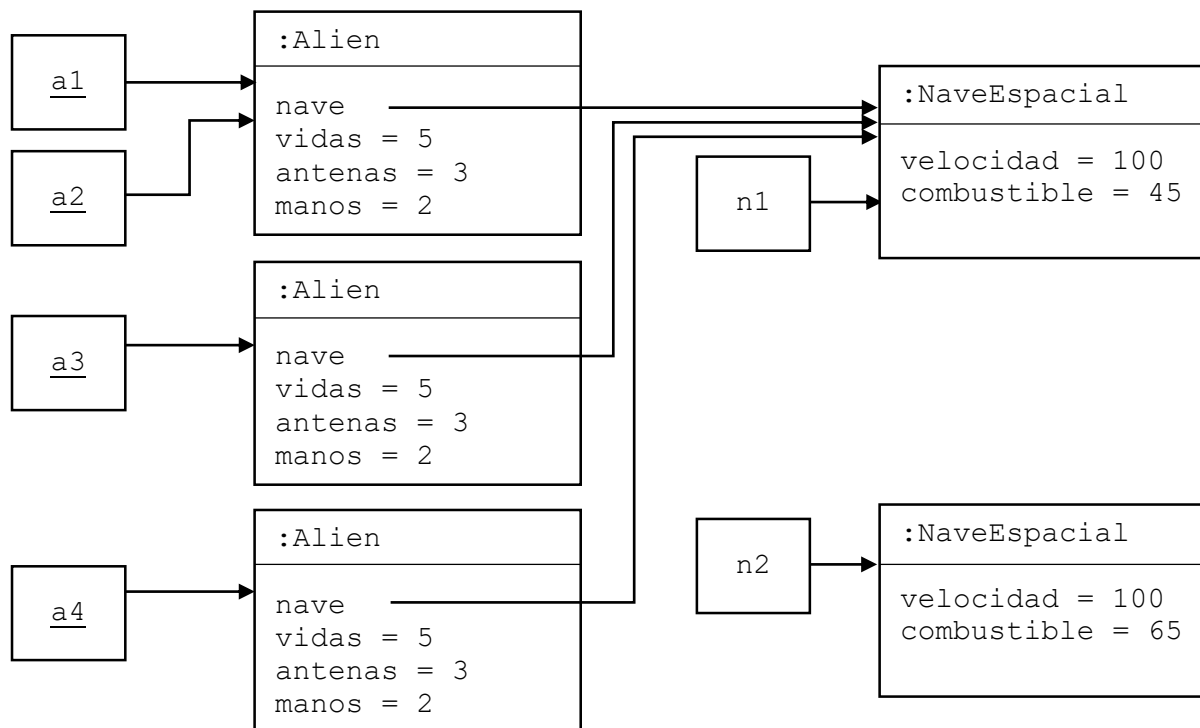


Figura 4

De modo que el operador relacional en las instrucciones:

```
System.out.println(a1==a2);
System.out.println(a1==a3);
System.out.println(a1==a4);
```

Computa `true`, `false` y `false`. Queda pendiente como ejercicio mostrar el diagrama de objetos luego de la ejecución del Tercer bloque y del Cuarto bloque considerando que `copy`, `clone` y `equals` se implementen **en forma superficial**.

El diseñador del videojuego puede especificar que los métodos `copy`, `clone` y `equals` se implementen **en profundidad**. En este caso el diseñador estableció también como requisito que el aliante está asociado a una nave. El código de **clone en profundidad** es:

```
public Alien clone(){
//Requiere que nave está ligada
NaveEspacial n = nave.clone();
int m = manos;
int a = antenas;
int v = vidas;
Alien c = new Alien(n,a,m);
c.establecerVidas(v);
return c;}

```

En esta implementación, se crea un clone del alien que recibe el mensaje, vinculado a un clone de la nave asociada al alien que recibe el mensaje. Es importante notar que si el atributo `nave` no estuviera ligado se produciría un error de ejecución. Sin embargo, si cada clase cumple con

las responsabilidades establecidas en el diseño, el alien solo recibe el mensaje `clone` si está ligado a una nave. .

La Figura 5 muestra el diagrama de objetos después de la ejecución del Primer bloque, considerando la implementación en profundidad de clone.

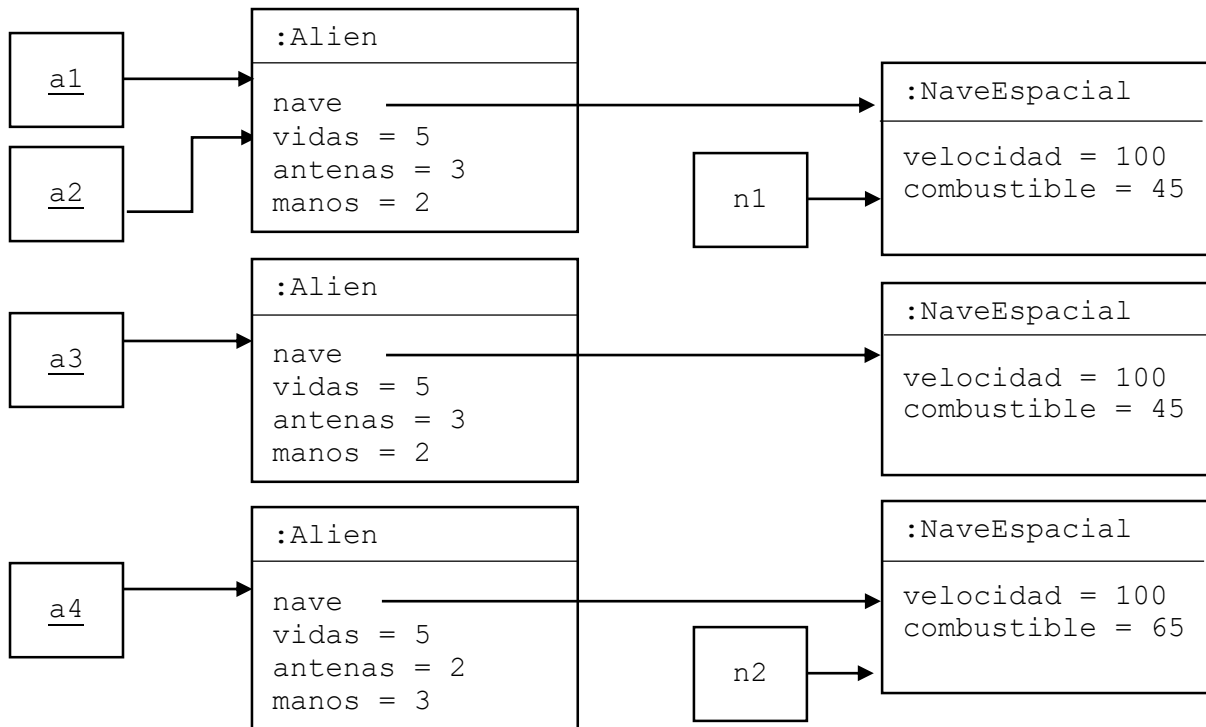


Figura 5

El objeto ligado a la variable `a3` mantiene en este caso una referencia a una nave que es un clone de la nave asociada al alien ligado a la variable `a1`.

La consulta `clone` definida en la clase `Alien` envía el mensaje `clone` al objeto ligado al atributo de instancia `nave`. Las clases `Alien` y `NaveEspacial` brindan métodos con el mismo nombre. **Es la clase del objeto que recibe el mensaje la que determina cuál de los métodos debe ejecutarse en cada caso.**

La implementación del comando **copy en profundidad** es:

```

public void copy (Alien a){
//Requiere a ligada y la nave de a ligada
    nave.copy(a.obtenerNave());
    vidas = a.obtenerVidas();
    antenas = a.obtenerAntenas();
    manos = a.obtenerManos();}

```

El diseñador estableció que el comando `copy` debe implementarse en profundidad y además tiene como requisito que el alien esté ligado a una nave. Después de la ejecución de la primera instrucción del Segundo bloque:

```
a4.copy(a1);
```

Se copia en el estado interno de la nave asociada al alien que recibe el mensaje, el estado interno de la nave del alien `a`, recibido como parámetro formal. La Figura 6 muestra el

diagrama de objetos luego de ejecutarse el Segundo bloque, para copy implementado en profundidad.

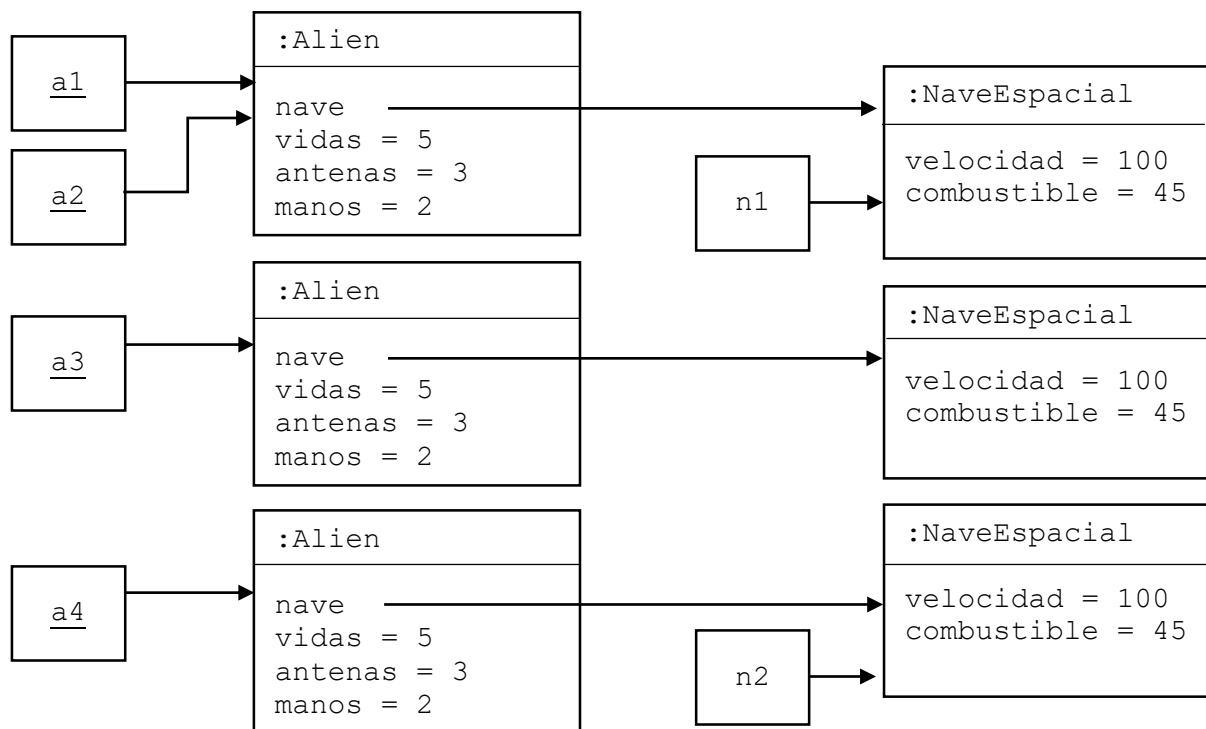


Figura 6

Es decir, la copia en profundidad modifica los valores de los atributos `vidas`, `antenas` y `manos`, y el estado interno de la nave asociada al alien que recibe el mensaje, pero no se modifica el valor del atributo instancia `nave` de la clase `Alien`.

La implementación de **equals en profundidad** es:

```

public boolean equals(Alien a){
//Requiere que nave está ligada
return Nave.equals(a.obtenerNave()) &&
    manos == a.obtenerManos() &&
    antenas == a.obtenerAntenas() &&
    vidas == a.obtenerVidas();}
  
```

La consulta `equals` definida en la clase `Alien` envía el mensaje `equals` al objeto ligado al atributo de instancia `Nave`. Nuevamente, es la clase del objeto que recibe el mensaje la que determina cuál de los métodos debe ejecutarse en cada caso.

Para esta implementación de la consulta `equals` y considerando el último diagrama de objetos, las instrucciones:

```

System.out.println(a1.equals(a3));
System.out.println(a1.equals(a4));
  
```

Muestran en consola:

```

true
true
  
```

Queda pendiente como ejercicio mostrar el diagrama de objetos luego de la ejecución del Tercer bloque y del Cuarto bloque considerando que `copy`, `clone` y `equals` se implementen **en profundidad** y requiere que el aliente está asociado a una nave.

Es posible que el diseñador especifique en el diagrama que una clase debe brindar, por ejemplo, el método `equals` implementado en forma superficial y también en profundidad. En este caso es necesario elegir un nombre diferente para cada consulta.

```
public boolean equalsS(Alien a){
    return nave == a.obtenerNave() &&
        manos == a.obtenerManos() &&
        antenas == a.obtenerAntenas() &&
        vidas == a.obtenerVidas();}
public boolean equalsP(Alien a){
    //Requiere que nave está ligada
    return nave.equals(a.obtenerNave()) &&
        manos == a.obtenerManos() &&
        antenas == a.obtenerAntenas() &&
        vidas == a.obtenerVidas();}
```

La clase `Alien` brinda dos métodos alternativos para decidir si dos objetos son equivalentes. El primero, más estricto, decide que dos aliens son equivalentes si están asociados a una misma nave. El segundo, considera que dos aliens son equivalentes si están asociados a naves equivalentes.

Clientes y proveedores de servicios

En el conjunto de clases que modelan una aplicación, algunas clases son **clientes** de los servicios provistos por otras clases **proveedoras**. Con frecuencia una misma clase puede cumplir ambos roles, es decir, ser cliente y proveedora a la vez.

Una clase que **usa** los servicios provistos por otra clase, es cliente de la clase que provee dichos servicios. Una clase que **tiene** atributos de otra clase, es cliente de las clases a las que corresponden esos atributos; estas últimas son proveedoras de servicios.

En particular una clase tester es una clase cliente de los servicios que brindan las clases que deben ser verificadas. Al implementar una clase tester se establece una dependencia entre esta clase y las que van a ser verificadas.

La clase cliente accede a la clase proveedora a través de su **interfaz**. La programación orientada a objetos propone **minimizar** la interfaz a través de la cual se comunican una clase cliente y una clase proveedora, de modo que se minimice también el impacto de los cambios.

En el control de signos vitales, cada objeto de clase `SignosVitales` está asociado a una instancia de clase `PresionArterial`. La clase `SignosVitales` **tieneUn** atributo de clase `PresionArterial`. La clase `SignosVitales` asume que cuando un objeto reciba el mensaje `alarma()` su atributo de instancia `presion` estará ligado. Varias mediciones de los signos vitales podrán estar asociados a registros de la presión arterial con los mismos valores, pero diferente identidad.

En el videojuego, cada objeto de clase `Alien` *puede* estar asociado a una instancia de clase `NaveEspacial`. La clase `Alien` **tieneUn** atributo de clase `NaveEspacial`. El diseñador estableció que los métodos `copy`, `equals` y `clone`, si son implementados en profundidad, requieren que el atributo `nave` esté ligado. Es decir, en este diseño la responsabilidad de verificar que la nave está ligada antes de enviar el mensaje `copy`, `equals` o `clone`, es de la clase cliente.

Los métodos `establecerNave` y `obtenerNave` de la clase `Alien`, generan también una relación de dependencia entre `Alien` y `NaveEspacial`.

La clase `Alien` es **cliente** de la clase **proveedora** `NaveEspacial`. La interfaz entre las clases `Alien` y `NaveEspacial` está formada por la signatura de los servicios públicos. La clase cliente `Alien` puede implementarse conociendo únicamente la interfaz de la clase proveedora `NaveEspacial`. La clase `NaveEspacial` se implementa desconociendo quienes van a ser sus clientes.

El atributo `nave` en la clase `Alien` mantiene una **referencia** a un objeto de clase `NaveEspacial`. En la realidad a modelar, probablemente varios aliens están asociados a una misma nave, en ejecución varias instancias de `Alien` referenciarán entonces a un mismo objeto de clase `NaveEspacial`.

El contrato entre la clase proveedora y la clase cliente

Entre una clase cliente y una clase proveedora de servicios, se establece un **contrato** que determina las **responsabilidades** de cada una. Las condiciones del contrato se especifican en la etapa de **diseño** del sistema e incluyen la **funcionalidad** y los **requisitos** inherentes a cada servicio. En la **implementación** es necesario interpretar las responsabilidades, los requisitos y la funcionalidad para reflejarlas en el código. Parte de la **verificación** consiste en analizar si cada clase cumple con el contrato.

Cuando una clase no cumple con su contrato pueden producirse errores de ejecución o aplicación que es necesario detectar y depurar. También puede ocurrir que el contrato no se haya especificado adecuadamente. En cualquier caso es fundamental diseñar una batería de casos de prueba que permitan detectar errores. El diseño de los casos de prueba puede hacerlo el diseñador del sistema o el responsable de la etapa de testado.

En sistemas desarrollados para resolver problemas críticos como controlar un avión o los semáforos de una ciudad, cada clase puede prever situaciones excepcionales, considerando que las demás clases no cumplan el contrato. En un sistema **robusto** cada clase establece controles que previenen errores provocados por otras clases, cuando estas no cumplen con sus responsabilidades. En este caso se utilizan los mecanismos provistos por el lenguaje para **manejar excepciones**. En este libro vamos a asumir que cada clase se implementa de acuerdo a la responsabilidades que se le asignan en el contrato.

Caso de Estudio: Facturas en Cuenta Corriente

*En una empresa de servicios se desea mantener información referida a las **facturas** imputadas a las cuentas corrientes de los **usuarios**. De cada **factura** se mantiene el **número**, el **monto** y la **cuenta corriente del usuario**. De cada **cuenta corriente** se mantiene el **nombre** del usuario y el **saldo**. Cuando se crea un objeto de clase `Factura` el constructor inicializa todos los atributos de instancia de acuerdo a los parámetros y actualiza el saldo de la cuenta corriente con el monto de la factura.*

Factura
<<Atributos de instancia>> nroFact: String montoFact: real ctaCteFact: Cta Cte
<<Constructor>>

Cta Cte
<<Atributos de instancia>> nombre: String saldo: real
<<Constructor>> Cta Cte (nom:String)

```
Factura(nro:String,m:real,c:
Cta_Cte)
<<Comandos>>
establecerCtaCteFact(c:Cta_Cte)
establecerMontoFact(m:real)
<<Consultas>>
obtenerNroFact():String
obtenerCtaCteFact():Cta_Cte
obtenerMontoFact():real
```

```
<<Comandos>>
establecerSaldo(m:real)
actualizarSaldo(m:real)
<<Consultas>>
obtenerNombre():String
obtenerSaldo():real
```

En la clase `Factura`:

`Factura(nro:String,m:real,c: Cta_Cte)` Requiere `nro` y `c` ligadas.

`establecerCtaCteFact(c:Cta_Cte)` Requiere `c` ligada.

En la clase `Cta_Cta`:

`Cta_Cte (nom:String)` Requiere `nom` ligada.

Las clases `Factura`, `String` y `Cta_Cte` están asociadas, la relación es de tipo **tieneUn**.

El comando `establecerCtaCteFact` de la clase `Factura` recibe como parámetro una variable de la `Cta_Cte`. La consulta `obtenerCtaCteFact` retorna una referencia a un objeto de la clase `Cta_Cte`. La asociación entre las clases `Factura` y `Cta_Cte` provoca también una relación de dependencia.

La clase `Factura` es la clase **cliente** de las clases `Cta_Cte` y `String`. La clase `Cta_Cte` es **proveedora** de servicios respecto a `Factura`, pero es también **cliente** de la clase `String`.

Las clases `Factura` y `Cta_Cte` son **proveedoras** de los servicios que van a ser usados por las clases que declaren variables de los tipos `Factura` y `Cta_Cte`. Estas clases deben asumir la responsabilidad de asegurar que una factura se crea ligada a un número y a una cuenta corriente. Análogamente se requiere que las clases cliente que envíen el mensaje `establecerCtaCteFact` a un objeto de clase `Factura`, lo hagan con una variable ligada como parámetro.

La clase `Factura` no controla que `nroFact` y `ctaCteFact` mantengan referencias ligadas, porque de acuerdo al contrato no es su responsabilidad realizar este control. En este diseño la clase `Factura` asume la responsabilidad de modificar el saldo de la cuenta corriente cuando se crea un objeto.

```
class Factura {
private String nroFact;
private float montoFact;
private Cta_Cte ctaCteFact;
public Factura (String n, float m,Cta_Cte c){
/*Crea una factura, guarda número, monto y cuenta corriente, y
actualiza el saldo de la cuenta corriente del usuario con el mismo
monto. Requiere que n y c estén ligadas */
nroFact = n;
montoFact = m;
ctaCteFact = c;
ctaCteFact.actualizarSaldo (m);}
}
```

El constructor envía el mensaje `actualizarSaldo` al objeto ligado a `ctaCteFact`, asumiendo que es una referencia ligada, porque así lo establece el requerimiento en el diagrama.

Después de la tercera asignación, las variables `c` y `ctaCteFact` están ligadas a un mismo objeto. Aunque el valor de la variable `c` no cambia, sí se modifica el estado interno del objeto referenciado por `c`. Decimos que la ejecución del constructor tiene un **efecto colateral**, no sólo inicializa el estado interno del objeto que se crea, sino que modifica también el estado interno de uno de los objetos que se recibe como parámetro.

```
class Ventas {
public static void test(){
    String num1 = new String("A-0001");
    String num2 = new String("A-0002");
    Cta_Cte cc = new Cta_Cte ("Gomez");
    ...
    Factura f1 = new Factura (num1,1500,cc);
    Factura f2 = new Factura (num2,1200,cc);
}
```

La clase `Ventas` es cliente de la clase `Factura`, el método `test` es responsable de asegurar que el tercer parámetro del constructor es una variable ligada, pero no de actualizar `saldo`. La Figura 7 muestra el diagrama de objetos después de la ejecución del segmento anterior.

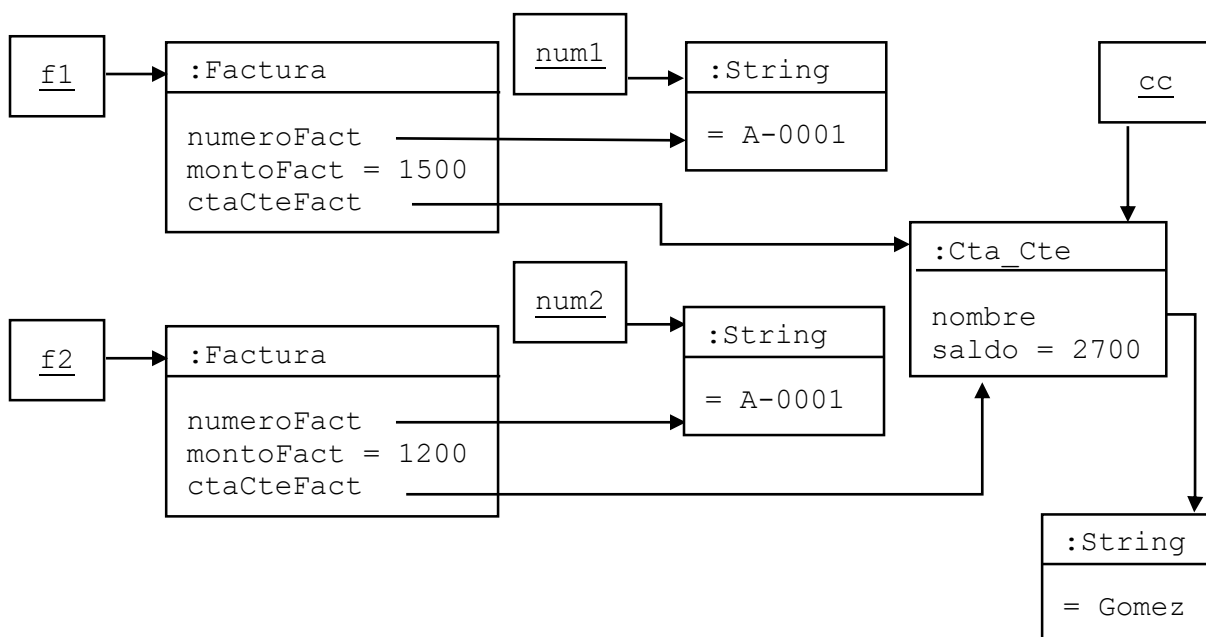


Figura 7

Consideremos el siguiente diseño alternativo:

*En una empresa de servicios se desea mantener información referida a las **facturas** imputadas a las cuentas corrientes de los **usuarios**. De cada **factura** se mantiene el **número**, el **monto** y la **cuenta corriente del usuario**. De cada **cuenta corriente** se mantiene el **nombre** del usuario y el **saldo**. Cuando se crea una factura, inmediatamente después se actualiza el saldo de la cuenta corriente.*

En esta alternativa de diseño, cada vez que se crea un objeto de clase `Factura` la responsabilidad de actualizar el saldo es de la clase cliente. El código del constructor para la segunda alternativa de diseño de `Factura` es:

```
class Factura {
private String nroFact;
private float montoFact;
```

```
private Cta_Cte ctaCteFact;
public Factura (String n, float m,Cta_Cte c){
/*Crea una factura, guarda número, monto y la cuenta corriente.
Requiere que n y c ligadas */
    nroFact = n;
    montoFact = m;
    ctaCteFact = c;
}
```

Nuevamente el método `test` de la clase `Ventas` **usa** los servicios provistos por `Factura`, pero además asume la responsabilidad de actualizar el saldo de la cuenta corriente del usuario.

```
class Ventas {
public static void test(){
    String num1 = new String("A-0001");
    String num2 = new String("A-0002");
    Cta_Cte cc = new Cta_Cte ("Gomez");
    Factura f1 = new Factura (num1,1500,cc);
    cc.actualizarSaldo (1500);
    Factura f2 = new Factura (num2,1200,cc);
    cc.actualizarSaldo (1200);}
}
```

El diagrama de objetos al completarse la ejecución del método `test`, es el mismo cualquiera sea la alternativa elegida.

El diseñador del sistema establece la **responsabilidad** de cada clase. El programador debe generar código adecuado para garantizar que cada clase cumple con sus responsabilidades. El responsable del testado busca errores de aplicación en base al contrato.

Cualquiera sea la decisión de diseño el código parcial de la clase `Cta_Cte` es:

```
class Cta_Cte {
//Atributos de Instancia
private String nombre;
private float saldo;
//Constructor
public Cta_Cte (String n){
    nombre =n;}
//Comandos
public void actualizarSaldo (float s) {
    saldo += s;}
...
}
```

Si el diseñador elige una de las alternativas y cambia de decisión una vez que las clases están implementadas, el cambio va a requerir modificar tanto la clase proveedora, como todas las clases que la usan. Una modificación de diseño que cambia las responsabilidades, afecta a la colección de clases asociadas. Si sólo se modifica una de las clases, por ejemplo la clase cliente, va a producirse un **error de aplicación**, que pasa desapercibido para el compilador.